

N-gram Language Modeling

Introduction to N-gram Language Models

Predicting words

The water of Walden Pond is beautifully ...

blue

green

clear

*refrigerator

*that

Language Models

Systems that can predict upcoming words

- Can assign a probability to each potential next word
- Can assign a probability to a whole sentence

Why word prediction?

It's a helpful part of language tasks

- Grammar or spell checking

Their are two midterms

~~Their~~ There are two midterms

Everything has improve

Everything has ~~improve~~ improved

- Speech recognition

I will be back soonish

I will be bassoon dish

Why word prediction?

It's how **large language models (LLMs)** work!

LLMs are **trained** to predict words

- Left-to-right (autoregressive) LMs learn to predict next word

LLMs **generate** text by predicting words

- By predicting the next word over and over again

Language Modeling (LM) more formally

Goal: compute the probability of a sentence or sequence of words W :

$$P(W) = P(w_1, w_2, w_3, w_4, w_5 \dots w_n)$$

Related task: probability of an upcoming word:

$$P(w_5 | w_1, w_2, w_3, w_4) \text{ or } P(w_n | w_1, w_2 \dots w_{n-1})$$

An LM computes either of these:

$$P(W) \quad \text{or} \quad P(w_n | w_1, w_2 \dots w_{n-1})$$

How to estimate these probabilities

Could we just count and divide?

$$P(\text{blue} | \text{The water of Walden Pond is so beautifully}) = \frac{C(\text{The water of Walden Pond is so beautifully blue})}{C(\text{The water of Walden Pond is so beautifully})}$$

No! Too many possible sentences!

We'll never see enough data for estimating these

How to compute $P(W)$ or $P(w_n | w_1, \dots, w_{n-1})$

How to compute the joint probability $P(W)$:

$P(\text{The, water, of, Walden, Pond, is, so, beautifully, blue})$

Intuition: let's rely on the Chain Rule of Probability

Reminder: The Chain Rule

Recall the definition of conditional probabilities

$$P(B|A) = P(A,B)/P(A) \quad \text{Rewriting: } P(A,B) = P(A) P(B|A)$$

More variables:

$$P(A,B,C,D) = P(A) P(B|A) P(C|A,B) P(D|A,B,C)$$

The Chain Rule in General

$$P(x_1, x_2, x_3, \dots, x_n) = P(x_1)P(x_2|x_1)P(x_3|x_1, x_2) \dots P(x_n|x_1, \dots, x_{n-1})$$

The Chain Rule applied to compute joint probability of words in sentence

$$\begin{aligned} P(w_{1:n}) &= P(w_1)P(w_2|w_1)P(w_3|w_{1:2}) \dots P(w_n|w_{1:n-1}) \\ &= \prod_{k=1}^n P(w_k|w_{1:k-1}) \end{aligned}$$

$P(\text{"The water of Walden Pond"}) =$

$P(\text{The}) \times P(\text{water}|\text{The}) \times P(\text{of}|\text{The water})$

$\times P(\text{Walden}|\text{The water of}) \times P(\text{Pond}|\text{The water of Walden})$

Markov Assumption

Simplifying assumption:



Andrei Markov

$P(\text{blue} | \text{The water of Walden Pond is so beautifully})$

$\approx P(\text{blue} | \text{beautifully})$

$$P(w_n | w_{1:n-1}) \approx P(w_n | w_{n-1})$$

Bigram Markov Assumption

$$P(w_{1:n}) \approx \prod_{k=1}^n P(w_k | w_{k-1})$$

Instead of:

$$\prod_{k=1}^n P(w_k | w_{1:k-1})$$

More generally, we approximate each component in the product

$$P(w_n | w_{1:n-1}) \approx P(w_n | w_{n-N+1:n-1})$$

Simplest case: Unigram model

$$P(w_1 w_2 \dots w_n) \approx \prod_i P(w_i)$$

Some automatically generated sentences from two different unigram models

To him swallowed confess hear both . Which . Of save on trail
for are ay device and rote life have

Hill he late speaks ; or ! a more to leg less first you enter

Months the my and issue of year foreign new exchange's September
were recession exchange new endorsed a acquire to six executives

Bigram model

$$P(w_i | w_1 w_2 \dots w_{i-1}) \approx P(w_i | w_{i-1})$$

Some automatically generated sentences from two different unigram models

Why dost stand forth thy canopy, forsooth; he is this palpable hit
the King Henry. Live king. Follow.

What means, sir. I confess she? then all sorts, he is trim, captain.

Last December through the way to preserve the Hudson corporation N.
B. E. C. Taylor would seem to complete the major central planners
one gram point five percent of U. S. E. has already old M. X.
corporation of living

on information such as more frequently fishing to keep her

Problems with N-gram models

- N-grams can't handle **long-distance dependencies**:

“The soups that I made from that new cookbook I bought yesterday were amazingly delicious.”

- N-grams don't do well at modeling new sequences with similar meanings

The solution: **Large language models**

- can handle much longer contexts
- because of using embedding spaces, can model synonymy better, and generate better novel strings

Why N-gram models?

A nice clear paradigm that lets us introduce many of the important issues for **large language models**

- **training** and **test** sets
- the **perplexity** metric
- **sampling** to generate sentences
- ideas like **interpolation** and **backoff**

N-gram Language Modeling

Introduction to N-grams

N-gram Language Modeling

Estimating N-gram Probabilities

Estimating bigram probabilities

The Maximum Likelihood Estimate

$$P(w_n | w_{n-1}) = \frac{C(w_{n-1} w_n)}{\sum_w C(w_{n-1} w)}$$

$$P(w_n | w_{n-1}) = \frac{C(w_{n-1} w_n)}{C(w_{n-1})}$$

An example

<s> I am Sam </s>

<s> Sam I am </s>

<s> I do not like green eggs and ham </s>

$$P(w_i | w_{i-1}) = \frac{c(w_{i-1}, w_i)}{c(w_{i-1})}$$

$$P(\mathbf{I} | \langle \mathbf{s} \rangle) = \frac{2}{3} = .67$$

$$P(\mathbf{Sam} | \langle \mathbf{s} \rangle) = \frac{1}{3} = .33$$

$$P(\mathbf{am} | \mathbf{I}) = \frac{2}{3} = .67$$

$$P(\langle \mathbf{/s} \rangle | \mathbf{Sam}) = \frac{1}{2} = 0.5$$

$$P(\mathbf{Sam} | \mathbf{am}) = \frac{1}{2} = .5$$

$$P(\mathbf{do} | \mathbf{I}) = \frac{1}{3} = .33$$

More examples:

Berkeley Restaurant Project sentences

can you tell me about any good cantonese restaurants close by

tell me about chez panisse

i'm looking for a good place to eat breakfast

when is caffe venezia open during the day

Raw bigram counts

Out of 9222 sentences

	i	want	to	eat	chinese	food	lunch	spend
i	5	827	0	9	0	0	0	2
want	2	0	608	1	6	6	5	1
to	2	0	4	686	2	0	6	211
eat	0	0	2	0	16	2	42	0
chinese	1	0	0	0	0	82	1	0
food	15	0	15	0	1	4	0	0
lunch	2	0	0	0	0	1	0	0
spend	1	0	1	0	0	0	0	0

Raw bigram probabilities

Normalize by unigrams:

i	want	to	eat	chinese	food	lunch	spend
2533	927	2417	746	158	1093	341	278

Result:

	i	want	to	eat	chinese	food	lunch	spend
i	0.002	0.33	0	0.0036	0	0	0	0.00079
want	0.0022	0	0.66	0.0011	0.0065	0.0065	0.0054	0.0011
to	0.00083	0	0.0017	0.28	0.00083	0	0.0025	0.087
eat	0	0	0.0027	0	0.021	0.0027	0.056	0
chinese	0.0063	0	0	0	0	0.52	0.0063	0
food	0.014	0	0.014	0	0.00092	0.0037	0	0
lunch	0.0059	0	0	0	0	0.0029	0	0
spend	0.0036	0	0.0036	0	0	0	0	0

Bigram estimates of sentence probabilities

$$\begin{aligned} P(< s > \text{ I want english food } < / s >) = \\ & P(\text{I} | < s >) \\ & \times P(\text{want} | \text{I}) \\ & \times P(\text{english} | \text{want}) \\ & \times P(\text{food} | \text{english}) \\ & \times P(< / s > | \text{food}) \\ & = .000031 \end{aligned}$$

What kinds of knowledge do N-grams represent?

$$P(\text{english}|\text{want}) = .0011$$

$$P(\text{chinese}|\text{want}) = .0065$$

$$P(\text{to}|\text{want}) = .66$$

$$P(\text{eat} \mid \text{to}) = .28$$

$$P(\text{food} \mid \text{to}) = 0$$

$$P(\text{want} \mid \text{spend}) = 0$$

$$P(i \mid \langle s \rangle) = .25$$

Dealing with scale in large n-grams

LM probabilities are stored and computed in log format, i.e. **log probabilities**

This avoids underflow from multiplying many small numbers

$$\log(p_1 \times p_2 \times p_3 \times p_4) = \log p_1 + \log p_2 + \log p_3 + \log p_4$$

If we need probabilities we can do one exp at the end

$$p_1 \times p_2 \times p_3 \times p_4 = \exp(\log p_1 + \log p_2 + \log p_3 + \log p_4)$$

Larger ngrams

4-grams, 5-grams

Large datasets of large n-grams have been released

- N-grams from Corpus of Contemporary American English (COCA) 1 billion words (Davies 2020)
- Google Web 5-grams (Franz and Brants 2006) 1 trillion words)
- Efficiency: quantize probabilities to 4-8 bits instead of 8-byte float

Newest model: infini-grams (∞ -grams) (Liu et al 2024)

- No precomputing! Instead, store 5 trillion words of web text in **suffix arrays**. Can compute n-gram probabilities with any n!

N-gram LM Toolkits

SRILM

- <http://www.speech.sri.com/projects/srilm/>

KenLM

- <https://kheafield.com/code/kenlm/>

N-gram Language Modeling

Estimating N-gram Probabilities